Devoir n°1 : Patrons de construction (INF461)

KOA ESSAMA PAULIN BRICE
Matricule : 25I2255

Introduction

Ce rapport présente deux exemples personnels inspirés des formes *Factory Method* et *Factory*. Le premier gère des documents administratifs ; le second gère des moyens de transport. Chaque exemple commence avec trois classes concrètes, puis est modifié par l'ajout d'une quatrième classe.

1 Exemples personnels initiaux : trois classes concrètes

Dans les deux cas, le client ne connaît le type concret à utiliser qu'à l'exécution.

1.1 Factory Method

Une seule classe `FabriqueDocument` reçoit le type souhaité et crée une `AttestationScolarite`, un `CertificatStage` ou un `ReleveNotes`. Le client dépend de l'interface `Document`.

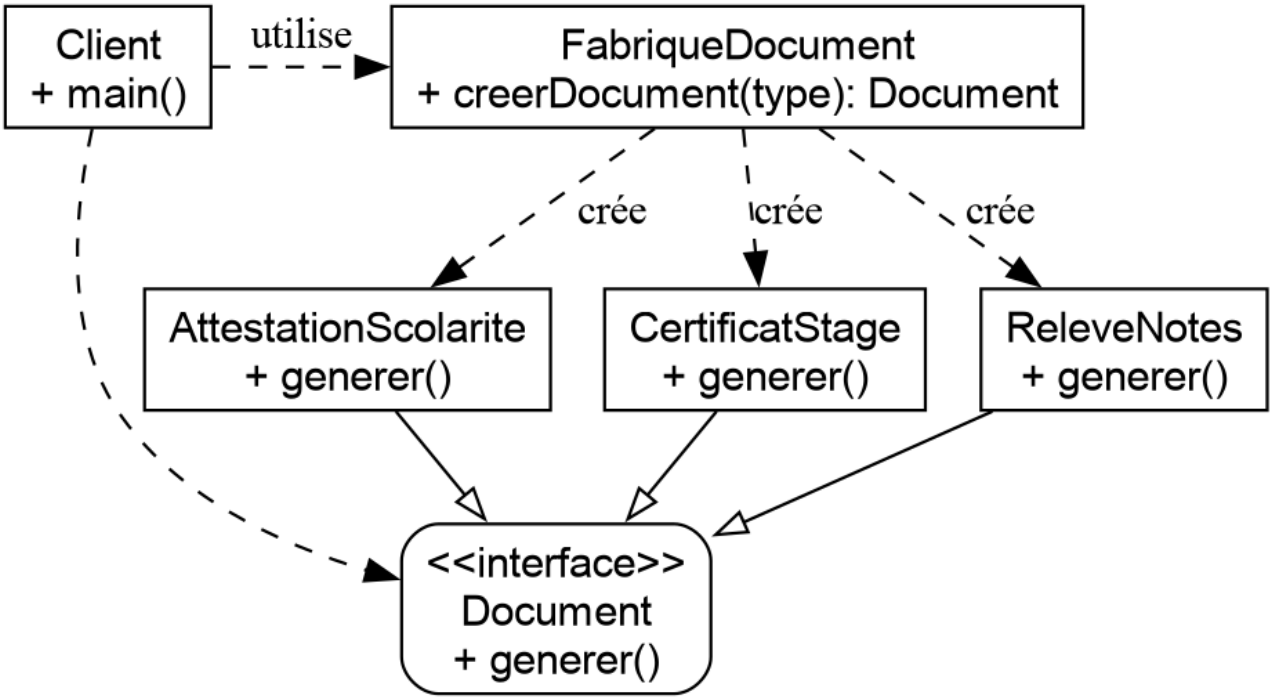


Figure 1: Factory Method – documents administratifs (3 documents)

1.2 Factory

`FabriqueTransport` définit la méthode de création. Les fabriques concrètes créent une `MotoTaxi`, un `TaxiVille` ou un `BusUrbain`.

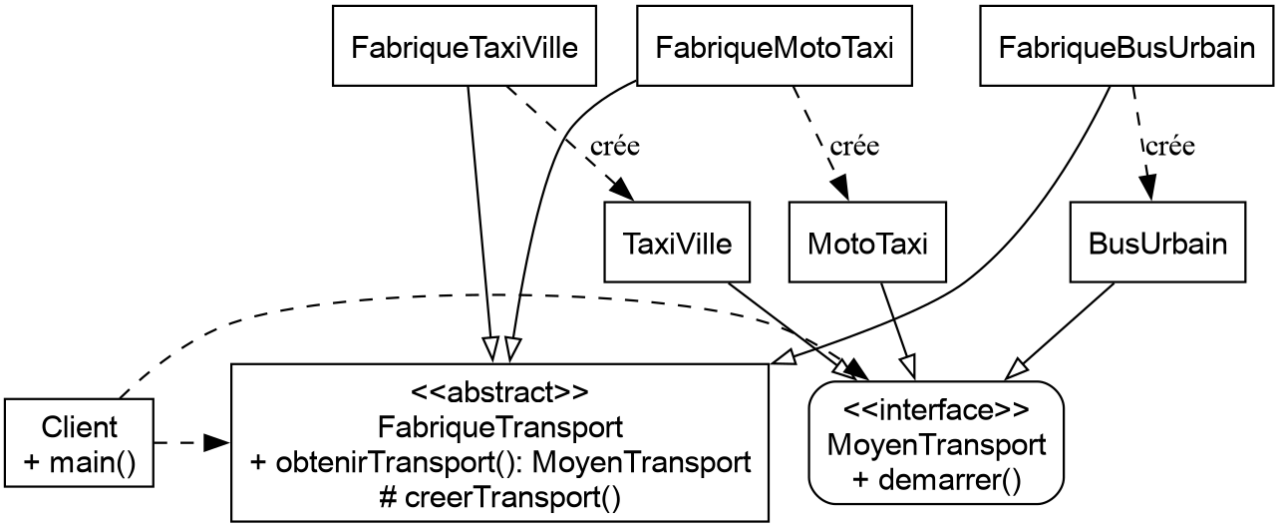


Figure 2: Factory – moyens de transport (3 transports)

2 Exemple modifié : quatre produits

La variante ajoute un quatrième élément à chacun des exemples.

2.1 Factory Method modifié

On ajoute `LettreRecommandation` et le cas `recommandation` dans `FabriqueDocument`. La fabrique centrale doit donc être modifiée.

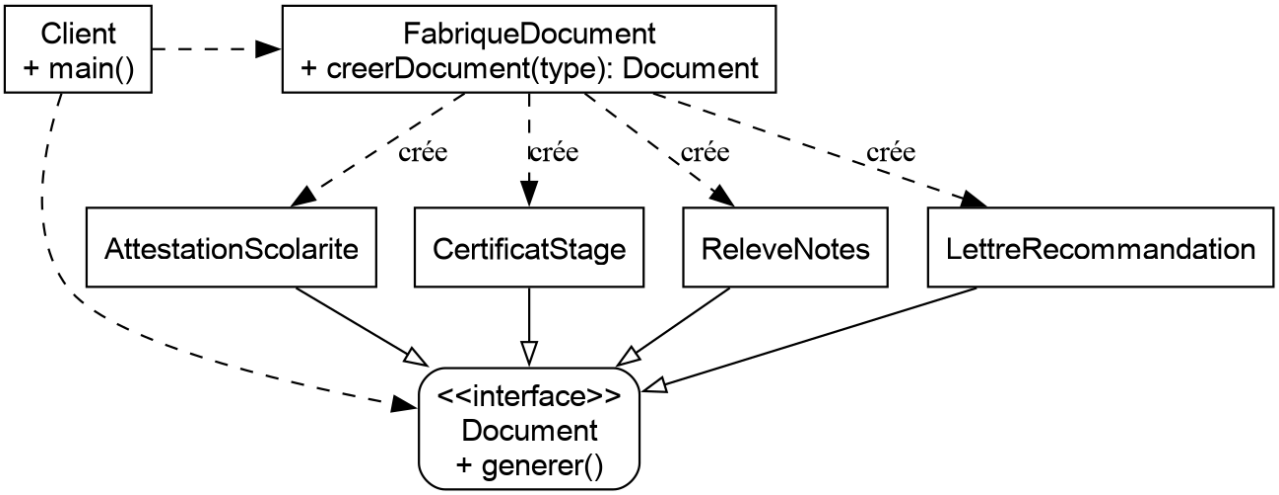


Figure 3: Factory Method – documents administratifs (4 documents)

2.2 Factory modifié

On ajoute `TrainInterurbain` et `FabriqueTrainInterurbain`. Les classes déjà présentes ne changent pas ; le client peut utiliser la nouvelle fabrique.

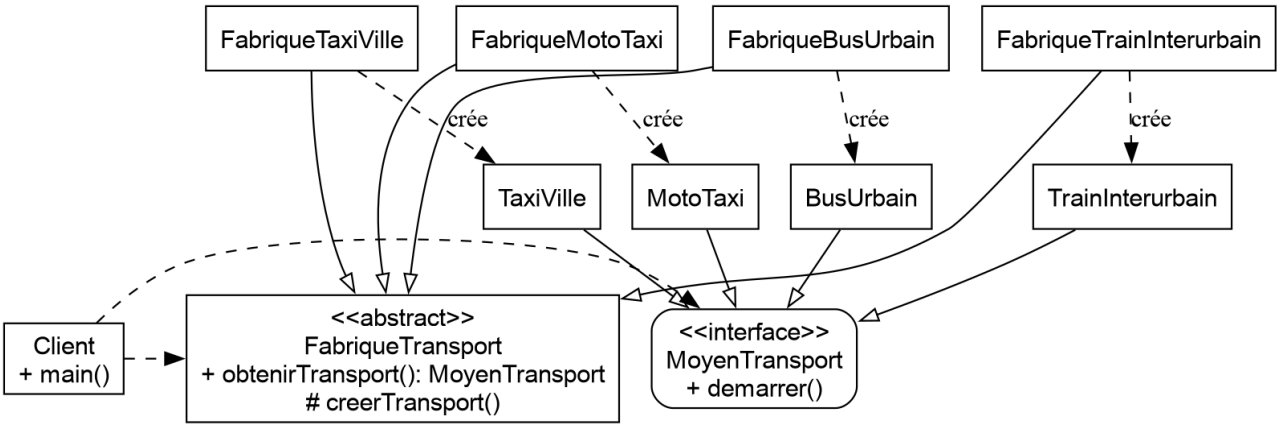


Figure 4: Factory – moyens de transport (4 transports)

3 Implémentation complète

Les quatre codes sources sont inclus ci-dessous. Ils peuvent être compilés directement avec Java.

3.1 Factory Method : documents administratifs (3 documents)

```
package factorymethod.troisproduits;

/** Exemple personnel : création de documents administratifs. */
public class Client {
    public static void main(String[] args) {
        FabriqueDocument fabrique = new FabriqueDocument();
        for (String type : new String[] {"attestation", "stage", "releve"}) {
            Document document = fabrique.creerDocument(type);
            System.out.println(document.generer());
        }
    }
}

interface Document {
    String generer();
}

class AttestationScolarite implements Document {
    public String generer() { return "Attestation de scolarité générée"; }
}

class CertificatStage implements Document {
    public String generer() { return "Certificat de stage généré"; }
}

class releveNotes implements Document {
    public String generer() { return "Relevé de notes généré"; }
}

class FabriqueDocument {
    public Document creerDocument(String type) {
        return switch (type.toLowerCase()) {
            case "attestation" -> new AttestationScolarite();
            case "stage" -> new CertificatStage();
            case "releve" -> new ReleveNotes();
            default -> throw new IllegalArgumentException("Type inconnu : " + type);
        };
    }
}
```

3.2 Factory Method : documents administratifs (4 documents)

```
package factorymethod.quatreproduits;

/** Variante : un quatrième document administratif est ajouté. */
public class Client {
    public static void main(String[] args) {
        FabriqueDocument fabrique = new FabriqueDocument();
        for (String type : new String[] {"attestation", "stage", "releve", "recommandation"}) {
            Document document = fabrique.creerDocument(type);
            System.out.println(document.generer());
        }
    }
}

interface Document {
    String generer();
}

class AttestationScolarite implements Document { public String generer() { return "Attestation de scolarité
     générée"; } }
class CertificatStage implements Document { public String generer() { return "Certificat de stage généré"; } }
class ReleveNotes implements Document { public String generer() { return "Relevé de notes généré"; } }
class LettreRecommandation implements Document { public String generer() { return "Lettre de recommandation
     générée"; } }

class FabriqueDocument {
    public Document creerDocument(String type) {
        return switch (type.toLowerCase()) {
            case "attestation" -> new AttestationScolarite();
```

```
            case "stage" -> new CertificatStage();
            case "releve" -> new releveNotes();
            case "recommandation" -> new LettreRecommandation();
            default -> throw new IllegalArgumentException("Type␣inconnu␣:␣" + type);
        };
    }
}
```

3.3 Factory : moyens de transport (3 transports)

```
package factory.troisproduits;

/** Exemple personnel : sélection de moyens de transport urbain. */
public class Client {
    public static void main(String[] args) {
        FabriqueTransport[] fabriques = {
            new FabriqueMotoTaxi(), new FabriqueTaxiVille(), new FabriqueBusUrbain()
        };
        for (FabriqueTransport fabrique : fabriques) {
            System.out.println(fabrique.obtenirTransport().demarrer());
        }
    }
}

interface MoyenTransport {
    String demarrer();
}

class MotoTaxi implements MoyenTransport { public String demarrer() { return "Moto-taxi␣prêt␣pour␣la␣course"; } }
class TaxiVille implements MoyenTransport { public String demarrer() { return "Taxi␣de␣ville␣prêt␣pour␣la␣course";
      } }
class BusUrbain implements MoyenTransport { public String demarrer() { return "Bus␣urbain␣prêt␣pour␣la␣course"; }
     }

abstract class FabriqueTransport {
    public MoyenTransport obtenirTransport() {
        return creerTransport();
    }
    protected abstract MoyenTransport creerTransport();
}

class FabriqueMotoTaxi extends FabriqueTransport {
    protected MoyenTransport creerTransport() { return new MotoTaxi(); }
}

class FabriqueTaxiVille extends FabriqueTransport {
    protected MoyenTransport creerTransport() { return new TaxiVille(); }
}

class FabriqueBusUrbain extends FabriqueTransport {
    protected MoyenTransport creerTransport() { return new BusUrbain(); }
}
```

3.4 Factory : moyens de transport (4 transports)

```
package factory.quatreproduits;

/** Variante : un train interurbain complète les trois transports initiaux. */
public class Client {
    public static void main(String[] args) {
        FabriqueTransport[] fabriques = {
            new FabriqueMotoTaxi(), new FabriqueTaxiVille(),
            new FabriqueBusUrbain(), new FabriqueTrainInterurbain()
        };
        for (FabriqueTransport fabrique : fabriques) {
            System.out.println(fabrique.obtenirTransport().demarrer());
        }
    }
}
```

```
interface MoyenTransport {
   String demarrer();
}

class MotoTaxi implements MoyenTransport { public String demarrer() { return "Moto-taxi prêt pour la course"; } }
class TaxiVille implements MoyenTransport { public String demarrer() { return "Taxi de ville prêt pour la course";
     } }
class BusUrbain implements MoyenTransport { public String demarrer() { return "Bus urbain prêt pour la course"; }
    }
class TrainInterurbain implements MoyenTransport { public String demarrer() { return "Train interurbain prêt pour
    le départ"; } }

abstract class FabriqueTransport {
   public MoyenTransport obtenirTransport() {
      return creerTransport();
   }
   protected abstract MoyenTransport creerTransport();
}

class FabriqueMotoTaxi extends FabriqueTransport { protected MoyenTransport creerTransport() { return new MotoTaxi
    (); } }
class FabriqueTaxiVille extends FabriqueTransport { protected MoyenTransport creerTransport() { return new
    TaxiVille(); } }
class FabriqueBusUrbain extends FabriqueTransport { protected MoyenTransport creerTransport() { return new
    BusUrbain(); } }
class FabriqueTrainInterurbain extends FabriqueTransport { protected MoyenTransport creerTransport() { return new
    TrainInterurbain(); } }
```

4 Organisation du code et dépôt GitHub

Le code est organisé en quatre packages :

- `factorymethod.troisproduits` : exemple du cours ;
- `factorymethod.quatreproduits` : variante à quatre produits ;
- `factory.troisproduits` : exemple du cours ;
- `factory.quatreproduits` : variante à quatre produits.

Le dépôt public est disponible à l'adresse : https://github.com/eliote-geeks/devoir-inf461-patrons-construction-koa-essama-paulin-brice.

5 Analyse des modèles génériques

Factory Method. Une seule fabrique choisit le document à l'aide d'un paramètre. Cette solution est simple lorsque le type est connu seulement à l'exécution. L'ajout de `LettreRecommandation` impose toutefois de modifier `FabriqueDocument`.

Factory. Une fabrique abstraite déclare `creerTransport()` et chaque sous-classe crée un moyen de transport concret. L'ajout du train consiste à créer `TrainInterurbain` et `FabriqueTrainInterurbain`, sans modifier les fabriques existantes. Cette solution respecte mieux le principe ouvert/fermé, mais nécessite davantage de classes.

6 Modèles génériques

Les deux modèles génériques étudiés servent de base aux exemples personnels précédents.

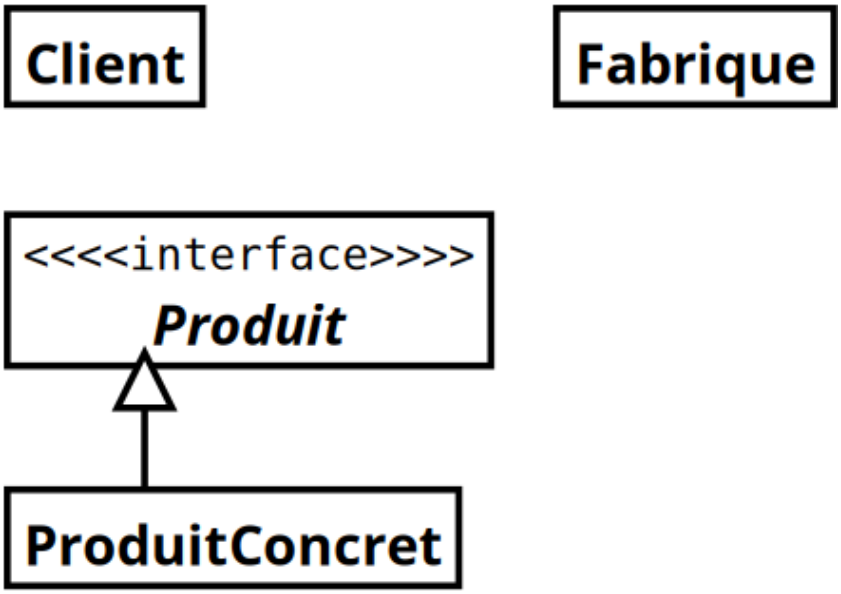


Figure 5: Modèle générique Factory Method

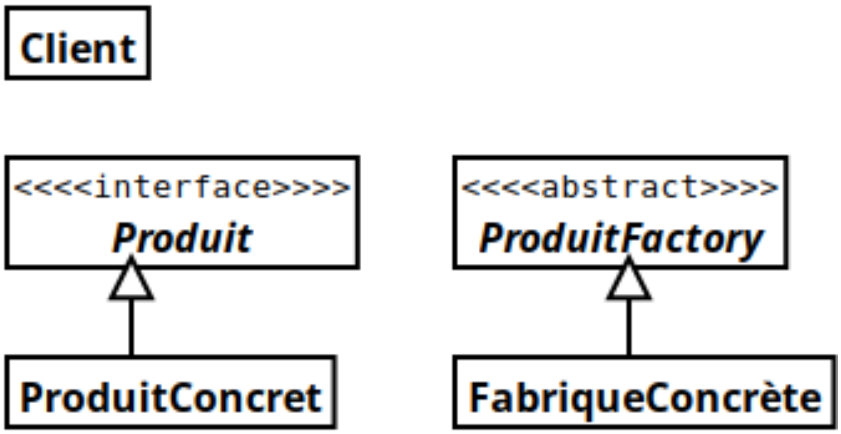


Figure 6: Modèle générique Factory